Java Backend Developer – 1st Round Interview (20 Questions)

1. How does HashMap work internally? Explain buckets, hashing & resizing.

Answer:

Internally, HashMap is an array of buckets. Each bucket holds either a linked list or a balanced tree of entries. When we put a key-value pair, HashMap calculates the hash of the key using hashCode(), then applies a hash function to reduce collisions, and finally finds the bucket index using (n - 1) & hash, where n is the array size.

If two keys map to the same bucket, collision happens. Before Java 8, collisions were handled using a linked list. From Java 8 onwards, if a bucket has more than 8 entries and capacity is large enough, it converts into a balanced tree (Red-Black Tree) for better performance, improving worst-case lookup from O(n) to O(log n).

Resizing happens when size exceeds capacity * loadFactor (default 0.75). When that threshold is crossed, the internal array size doubles, and all existing entries are rehashed and redistributed. That’s why frequent resizing is expensive and why we should set initial capacity properly in high-traffic systems like caching product details in e-commerce.

Example:

```
Map<String, String> map = new HashMap<>(16, 0.75f);
map.put("orderId", "ORD123");
```

Here 16 is initial capacity, and resizing happens after 12 entries.

2. What is the difference between HashMap and ConcurrentHashMap?

Answer:

The main difference is thread safety.

HashMap is not thread-safe. If multiple threads modify it at the same time, it can lead to data inconsistency or even infinite loops during resizing in older Java versions.

ConcurrentHashMap is designed for concurrent access. Instead of locking the entire map, it uses finer-grained locking or CAS (Compare-And-Swap) operations. In Java 8+, it uses synchronized blocks only at bucket level when necessary, allowing multiple threads to read and write concurrently with high performance.

Also, ConcurrentHashMap does not allow null keys or null values, because null creates ambiguity in concurrent scenarios.

In a real-world scenario, imagine a ticket booking system caching seat availability. If multiple users are checking and updating seats, using HashMap can break under concurrency. ConcurrentHashMap ensures thread-safe reads and updates without blocking the whole structure.

Example:

```
ConcurrentHashMap<String, Integer> seatMap = new ConcurrentHashMap<>();
seatMap.put("A1", 1);
seatMap.computeIfPresent("A1", (k, v) -> v - 1);
```

3. How does JVM memory structure work? (Heap, Stack, Metaspace)

Answer:

JVM memory is divided mainly into Heap, Stack, and Metaspace, and each serves a different purpose.

Stack memory is thread-specific. Every time a method is called, a stack frame is created containing local variables, method parameters, and references to objects. Once the method finishes, that frame is removed. So stack is fast and automatically managed. For example, in a payment API, when calculateTotal() is called, its local variables live in the stack.

Heap memory is shared across all threads. All objects created using new are stored here. This is where garbage collection works. If we create a new Order object in an e-commerce application, it lives in the heap until it becomes unreachable.

Metaspace stores class-level metadata like class definitions, method metadata, static variables, etc. Earlier it was called PermGen, but now it grows dynamically (by default limited by system memory).

So in short:

- Stack → method execution memory (per thread)
- Heap → objects and shared data
- Metaspace → class metadata

Understanding this is important because most memory-related production issues are heap-related.

4. What causes OutOfMemoryError in production? How would you debug it?

Answer:

OutOfMemoryError usually happens when the JVM cannot allocate memory in the heap or Metaspace.

Common real causes in production:

1. Memory leaks — objects are not getting garbage collected because references are still held. For example, storing user sessions in a static Map without removing expired ones.
2. Large collections growing infinitely — like caching without eviction.
3. High traffic creating too many objects.
4. Too small heap size configuration.
5. Metaspace overflow due to dynamic class generation (common in proxy-heavy frameworks).

How I would debug it step by step:

First, enable heap dump on OOM using:

```
-XX:+HeapDumpOnOutOfMemoryError
```

Then analyze the heap dump using tools like Eclipse MAT or VisualVM.

I would look for:

- Which objects consume most memory
- Whether there are unexpected large collections
- Reference chains to identify why objects are not garbage collected

In one real scenario, I saw a memory leak because we were storing API responses in a static HashMap without cleanup. Over time, heap kept growing until OOM happened.

So debugging is about identifying what object is growing abnormally and why GC is unable to clean it.

5. How do you make a class thread-safe? Give a real scenario.

Answer:

Making a class thread-safe means ensuring its state remains consistent even when accessed by multiple threads simultaneously.

There are multiple approaches:

1. Make the class immutable.
2. Use synchronized methods or blocks.
3. Use concurrent collections.
4. Use atomic classes like AtomicInteger.

The best approach depends on the use case.

For example, suppose we are building a wallet service where multiple users can add money concurrently:

Bad implementation:

```
class Wallet {
    private int balance;

    public void add(int amount) {
        balance += amount;
    }
}
```

If two threads call add() at the same time, updates can be lost.

Thread-safe version using synchronization:

```
class Wallet {
    private int balance;

    public synchronized void add(int amount) {
        balance += amount;
    }
}
```

Or better, using AtomicInteger:

```
class Wallet {
    private AtomicInteger balance = new AtomicInteger(0);

    public void add(int amount) {
        balance.addAndGet(amount);
```

```
    }
}
```

In real systems like payment or inventory management, we often combine database-level locking with application-level thread safety to avoid race conditions.

So thread safety is about controlling access to shared mutable state using proper synchronization or immutability principles.

6. How does Spring Bean lifecycle work?

Answer:

When Spring starts, it scans the classpath, identifies beans, and creates objects inside the ApplicationContext. That's the beginning of the bean lifecycle.

Step by step what happens:

First, Spring instantiates the bean using the constructor.

Second, it performs dependency injection. If the bean has @Autowired fields or constructor parameters, Spring resolves and injects those dependencies.

Third, Spring calls aware interfaces if implemented, like BeanNameAware or ApplicationContextAware. This allows the bean to know about its container.

Fourth, BeanPostProcessor runs before initialization. This is important because Spring AOP and proxies are created here.

Then initialization happens:

- If the bean has @PostConstruct, that method is called.
- If it implements InitializingBean, afterPropertiesSet() is called.
- If initMethod is defined in configuration, that method runs.

Now the bean is ready to serve requests.

When the application shuts down:

- @PreDestroy is called.
- destroy() from DisposableBean is called.
- custom destroy methods are executed.

In a real-world scenario, imagine a database connection pool bean. During initialization, it establishes connections. During destroy, it closes them cleanly. That's lifecycle management in action.

7. How does Dependency Injection work internally in Spring?

Answer:

Dependency Injection in Spring is handled by the IoC container.

When Spring starts, it scans for components using annotations like @Component, @Service, etc. It creates BeanDefinitions for each class.

When a bean is required, Spring:

1. Creates the object using reflection.
2. Inspects its constructor, fields, or setter methods.
3. Resolves dependencies from the container.
4. Injects them.

For example:

```
@Service
class PaymentService {
    private final OrderService orderService;

    public PaymentService(OrderService orderService) {
        this.orderService = orderService;
    }
}
```

Spring sees that PaymentService needs OrderService. It checks the container, finds OrderService bean, and injects it through the constructor.

Internally, it uses reflection and BeanFactory to resolve dependencies. If multiple implementations exist, it uses @Primary or @Qualifier to decide.

In real systems, this removes tight coupling. Instead of PaymentService creating new OrderService(), Spring manages object creation, which improves testability and modularity.

8. What actually happens when you use @Transactional?

Answer:

@Transactional works using Spring AOP and proxies.

When we annotate a method with @Transactional, Spring does not modify the method directly. Instead, it creates a proxy around the bean.

When the method is called from outside the bean:

1. The proxy intercepts the call.
2. It starts a database transaction using the configured transaction manager.
3. It executes the actual method.
4. If the method completes successfully, it commits the transaction.
5. If a RuntimeException occurs, it rolls back.

For example:

```
@Transactional
public void transferMoney(Long from, Long to, double amount) {
    debit(from, amount);
    credit(to, amount);
}
```

In a banking example, if debit succeeds but credit fails, the entire transaction is rolled back, ensuring data consistency.

Important point: self-invocation doesn’t work. If a method inside the same class calls a @Transactional method, the proxy is bypassed and transaction won’t start.

So internally, it’s proxy-based transaction management using AOP, not magic inside the method itself.

9. Difference between @Component, @Service and @Repository?

Answer:

Technically, all three are specializations of @Component. So from Spring's perspective, they all register a bean in the container.

The difference is mainly semantic and behavioral in one case.

@Component is a generic stereotype annotation. We use it when the class doesn't clearly belong to service or repository layer.

@Service is meant for business logic classes. It doesn't add extra behavior internally, but it improves readability and communicates intent. When I see @Service, I immediately know this class contains business rules, like payment calculation or order processing.

@Repository is used for data access layer. The important difference is that Spring automatically translates database-specific exceptions into Spring's DataAccessException hierarchy when we use @Repository. That's done through PersistenceExceptionTranslationPostProcessor.

In a layered e-commerce system:

- Controller handles HTTP
- @Service contains business logic like applyDiscount()
- @Repository interacts with database like findOrderById()

So the main difference is semantic clarity and exception translation support in @Repository.

10. How would you handle global exception handling in Spring Boot?

Answer:

In production systems, we should not return raw stack traces. Instead, we create a centralized exception handling mechanism.

Spring Boot provides @ControllerAdvice for global exception handling.

Example:

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<String> handleNotFound(ResourceNotFoundException
ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
                .body(ex.getMessage());
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<String> handleGeneric(Exception ex) {
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
                .body("Something went wrong");
    }
}
```

Now whenever ResourceNotFoundException is thrown anywhere in controllers, this handler captures it.

In real systems, instead of returning plain strings, we return a structured error response like:

- timestamp
- status code
- error message
- path

This ensures consistent API responses across the system.

In a banking API, if account is not found, instead of 500 error, we return 404 with proper message. That improves client-side handling and debugging.

11. What is lazy vs eager loading in JPA? When can it cause performance issues?

Answer:

Lazy and eager loading define when related entities are fetched.

Eager loading means related data is fetched immediately along with the parent entity.

Lazy loading means related data is fetched only when we access it.

Example:

```
@OneToMany(mappedBy = "order", fetch = FetchType.LAZY)
private List<OrderItem> items;
```

If fetch is LAZY, when we fetch Order, OrderItems are not loaded until getItems() is called.

Now performance issue:

1. N+1 problem. Suppose we fetch 100 orders, and for each order we access items. If lazy loading is used without proper join fetch, JPA may execute 1 query for orders and 100 queries for items. That becomes a performance bottleneck.
2. Eager loading can also cause problems. If an entity has many relationships and all are eager, one simple query may fetch a huge amount of unnecessary data.

In real-world APIs, we carefully design fetch strategies and often use JPQL with join fetch or DTO projections to control what gets loaded.

So lazy gives control and efficiency, but if misused, it can cause N+1 query issues. Eager gives convenience but may overload queries unnecessarily.

12. How do you handle concurrent updates to the same database row?

Answer:

When multiple users try to update the same row, like two people booking the last seat in a flight, we need to prevent data inconsistency.

There are mainly two approaches: optimistic locking and pessimistic locking.

Optimistic locking assumes conflicts are rare. We add a version column in the table.

Example:

```
@Version
private Long version;
```

When JPA updates the row, it checks whether the version is the same as when it was read. If another transaction has already modified it, the update fails with OptimisticLockException. Then we can retry or show a proper error message.

This is good for systems like profile updates where conflicts are rare.

Pessimistic locking locks the row at database level.

Example:

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
```

This prevents others from modifying the row until the transaction completes. It's useful in high-conflict scenarios like inventory deduction in flash sales.

So in real systems:

- Low conflict → optimistic locking
- High conflict, critical consistency → pessimistic locking

13. Explain ACID properties with a banking transaction example.

Answer:

ACID ensures database reliability.

Let’s take a banking transfer of 1000 from Account A to Account B.

Atomicity means the transaction either fully completes or fully fails. If money is deducted from A but credit to B fails, the whole transaction rolls back.

Consistency means the database moves from one valid state to another. Total balance before and after transaction remains logically correct.

Isolation means concurrent transactions don’t interfere incorrectly. If two transfers happen at the same time, intermediate states are not visible to others.

Durability means once the transaction commits, it is permanently stored, even if the system crashes.

So ACID ensures that financial operations remain reliable even under concurrency and failures.

14. How do you optimize a slow SQL query?

Answer:

First, I would not guess. I would check the execution plan using EXPLAIN.

Step by step approach:

1. Check if proper indexes exist on WHERE, JOIN, and ORDER BY columns.
2. Avoid SELECT * — fetch only required columns.
3. Ensure joins are optimized and unnecessary joins are removed.
4. Check for full table scans.
5. If data is large, consider pagination.
6. Normalize or denormalize depending on use case.
7. Check database statistics and update them if outdated.

For example, if we search orders by customerId frequently, adding an index:

```
CREATE INDEX idx_customer ON orders(customer_id);
```

In real production cases, 80% of performance issues are fixed by proper indexing and query rewriting.

Sometimes we also use caching like Redis if the query result doesn’t change frequently.

15. How would you design pagination & sorting in a REST API?

Answer:

Pagination prevents returning huge datasets at once.

In REST, we usually pass parameters like:

```
/orders?page=0&size=10&sort=createdAt,desc
```

In Spring Boot, we can use Pageable:

```
@GetMapping("/orders")
public Page<Order> getOrders(Pageable pageable) {
    return orderRepository.findAll(pageable);
}
```

This automatically handles:

- limit
- offset
- sorting

Best practices:

- Always enforce max page size.
- Return metadata like total pages and total elements.
- Use cursor-based pagination for very large datasets (better performance than offset-based).

In e-commerce listing pages, pagination ensures we don't load 10,000 products at once, which improves performance and user experience.

Proper pagination design prevents memory pressure and improves response time significantly.

16. How does JWT authentication work step by step?

Answer:

JWT authentication is stateless, which makes it ideal for microservices and distributed systems.

Step by step flow:

First, user logs in with username and password.

Second, server validates credentials against database.

Third, if valid, server generates a JWT token. The token has three parts:

- Header (algorithm and type)
- Payload (claims like userId, roles, expiry time)
- Signature (to verify integrity)

The token is signed using a secret key or private key.

Example structure:

header.payload.signature

Fourth, the server sends this token back to the client.

Now for every subsequent request, the client sends the token in the Authorization header:

```
Authorization: Bearer <token>
```

On each request:

1. Server extracts the token.
2. Verifies signature.
3. Checks expiry.
4. Extracts user details from payload.
5. Sets authentication in SecurityContext.

No database lookup is required if token is valid.

In a real-world example like an e-commerce app, after login, user can browse products, place orders, etc., without session storage on the server. That improves scalability because the server doesn’t store session state.

The only caution is proper token expiration and secure secret management.

17. If your API is slow under load, how would you identify the bottleneck?

Answer:

First, I would not assume it's the code. I would approach it systematically.

Step one: Monitor metrics.
Check CPU usage, memory usage, thread pool utilization, database connections, and response time using tools like Prometheus, Grafana, or APM tools.

Step two: Identify whether it's CPU-bound, memory-bound, I/O-bound, or DB-bound.

If CPU is high → maybe heavy computation or too many threads.
If DB queries are slow → check query execution plan.
If thread pool is exhausted → increase pool size or optimize blocking calls.
If external API calls are slow → measure network latency.

Step three: Take thread dump during high load and analyze blocked threads.

Step four: Check GC logs. Frequent full GC can slow down API significantly.

In a real scenario, I once saw an API slow down because connection pool size was 10 but traffic required 50 concurrent DB connections. Increasing pool size improved performance immediately.

So identifying bottleneck is about measurement first, optimization later.

18. REST vs Messaging (Kafka), when would you choose which?

Answer:

REST is synchronous communication. Client sends request and waits for response.

Messaging like Kafka is asynchronous. Producer sends message to topic and doesn't wait for consumer to process it.

I choose REST when:

- Immediate response is required.
- Client needs confirmation instantly.

  Example: Checking account balance.

I choose Kafka when:

- Event-driven architecture.
- Loose coupling between services.
- High throughput required.

  Example: Order placed → inventory update → email notification → analytics event. Instead of chaining REST calls, we publish an event to Kafka and multiple consumers process it independently.

REST is simpler and easier to debug.
Kafka is better for scalability and decoupling.

In microservices, usually we combine both. REST for synchronous queries, Kafka for background processing.

19. How would you Dockerize a Spring Boot application?

Answer:

First, package the Spring Boot app as a JAR using:

```
mvn clean package
```

Then create a Dockerfile:

```
FROM openjdk:17-jdk-slim
WORKDIR /app
COPY target/app.jar app.jar
ENTRYPOINT ["java","-jar","app.jar"]
```

Then build image:

```
docker build -t spring-app
```

Run container:

```
docker run -p 8080:8080 spring-app
```

In production, we also:

- Use multi-stage builds to reduce image size
- Externalize configuration using environment variables
- Set memory limits properly
- Use health checks

In microservices environments like Kubernetes, Docker ensures consistent deployment across environments.

20. If two microservices fail during communication, how do you handle fault tolerance?

Answer:

In distributed systems, failures are normal, not exceptions. So we design for failure.

First, use timeouts. Never allow infinite waiting.

Second, implement retries with exponential backoff.

Third, use circuit breaker pattern. If a service keeps failing, circuit opens and stops calling it temporarily. This prevents cascading failures. Resilience4j is commonly used in Spring Boot.

Fourth, fallback mechanism. For example, if recommendation service fails, we can return default recommendations instead of failing the whole request.

Fifth, use bulkhead pattern to isolate resources.

In real-world example, suppose payment service calls fraud detection service. If fraud service is down, instead of blocking payments entirely, we can:

- Retry 2 times
- If still failing, log event and allow low-risk transactions temporarily

So fault tolerance includes:

- Retry
- Circuit breaker
- Timeout
- Fallback
- Proper monitoring

That’s how we design resilient microservices.